WWE ANAYLSIS

Project Working Guide · Tech Stack · Interview Prep

Premium Live Event Analytics Platform (2000–2027)

Live: https://wwe-anaylsis.vercel.app
Repo: https://github.com/alirazach476/wwe_anaylsis

Use this pack to walk interviewers through **what you built**, **how data flows**, **why each technology was chosen**, and **crisp answers** for system-design and data-engineering questions.

1. Project Overview — How It Works

WWE Anaylsis is an end-to-end analytics product for WWE Premium Live Events (PLEs). It collects and validates historical event/match data, loads a dimensional warehouse, materializes analytics marts, serves them through a FastAPI layer, and renders a broadcast-style React dashboard with event-logo theming, wrestler/event deep dives, championship analytics, and curated historical landmarks.

Product surfaces

Page	What it shows
Home / Overview	Catalog KPIs, attendance trends, chart gallery moved from Events
Events + Detail	Poster cards, filters, match card, duration charts, logo theme
Matches	Duration / finish-method analytics with sticky custom filters
Wrestlers + Detail	Full deduped roster, Man/Woman avatars, career stats
Championships	PLE title landscape including Crown Jewel featured belts
Timeline	Curated landmark records (e.g. Cena 17 world titles, Reigns 1,316 days)

End-to-end data flow

`Registry / MediaWiki API → Clean / Validate → Dimensional Warehouse → Marts CSV → FastAPI → React (Vite) → Vercel`

- **Raw / inventory:** event series registry + collectors (Wikipedia / MediaWiki), eligibility rules excluding weekly TV & house shows.
- **Cleaning:** normalize names, dedupe wrestlers, flag conflicts; null attendance is never coerced to 0.
- **Warehouse:** dimensional model (events, matches, participants, championships) in SQLite/Postgres.
- **Marts:** pre-aggregated CSVs for overview, attendance, wrestlers, matches, Rumble/MITB, championships.
- **API:** FastAPI reads marts (+ runtime export fallbacks) — dashboard never invents metrics client-side.
- **UI:** React pages consume `/api/*`; Plotly charts; themes sampled from event logo palettes.
- **Deploy:** Vercel static frontend + Python serverless ASGI (`api/index.py`) with marts/exports bundled via `includeFiles`.

Design constraints (say these in interviews)

- No fabricated historical rows — curated Timeline records are labeled as landmarks when marts diverge from official marks.
- Championship analytics = PLE title changes only (not weekly TV defenses).
- Dashboard metrics come from API/marts only.
- Serverless FS is read-only — settings mkdir is guarded; exports resolve to `runtime_exports` on Vercel.

2. Tech Stack

Layer	Technology	Why it fits
Language (BE)	Python 3.11+	Pandas + FastAPI ecosystem; strong for ETL
API	FastAPI + Uvicorn / ASGI	Typed routes, fast cold starts, Vercel ASGI
Validation	Pydantic / pydantic-settings	Config + schema contracts for inventory
Dataframes	Pandas (+ NumPy)	Mart materialization & analytics transforms
ORM / DB	SQLAlchemy, SQLite / Postgres	Local SQLite; Compose Postgres optional
Transform SQL	dbt (optional path)	Documented SQL marts alongside Python
Collection	requests / httpx, BS4, lxml	MediaWiki + HTML parsing, retries (tenacity)
Frontend	React 18 + TypeScript + Vite	SPA speed, typed UI, modern DX
Charts	Plotly (react-plotly.js)	Interactive analytics visuals
Routing	React Router 6	Event/wrestler deep links
Theming	Logo-derived CSS variables	Event-specific broadcast look
Deploy	Vercel (static + Python fn)	One project for UI + API
Containers	Docker Compose	Postgres + API + frontend local parity
Orchestration	Airflow DAG (optional)	Warehouse refresh scheduling
Tests	Pytest	Registry, eligibility, dedupe, warehouse

Key API endpoints (talking points)

- `/health` — marts present? warehouse sqlite?
- `/api/analytics/overview`, `/attendance`, `/matches`, `/championships`
- `/api/events`, `/api/wrestlers` (full roster + dedupe), `/api/matches`
- `/api/analytics/historical-records` — curated Timeline landmarks
- `/api/filters/meta` — drives sticky FilterSelect portals

Repo map (30-second tour)

`api/ → Vercel entry | frontend/ → React app | src/wwe_data/ → collectors, warehouse, API`
`data/warehouse/marts + runtime_exports → production data | config/ → series registry & eligibility`
`docs/ → architecture & dictionary | scripts/ → one-offs (e.g. Crown Jewel)`

3. Architecture Deep Dive

Dimensional thinking

Facts (matches, title changes) are separated from dimensions (events, wrestlers, venues, brands, dates). Marts are **denormalized read models** optimized for dashboard queries so the API stays simple and cheap on serverless.

Vercel production shape

- Build: `cd frontend && npm run build` → `frontend/dist`
- Rewrites: `/api/*` and `/health` → Python ASGI; SPA fallback to `index.html`
- Bundle: `includeFiles` packs `src/**`, marts, and `runtime_exports` (because `data/exports` is gitignored)
- Assets: event logos live under `frontend/public/logos/files` (must not be ignored by `.vercelignore`)

Hard problems you solved (story bank)

- **Wrestler identity:** same performer under aliases → dedupe + full roster listing (not capped samples).
- **Attendance honesty:** exclude zeros / missing from averages; yearly avg chart instead of sum-as-avg mistakes.
- **Official vs warehouse records:** Timeline uses curated landmarks where PLE-scoped marts undercount all-time marks.
- **UX:** sticky filter strip + portal-based FilterSelect to avoid overflow clipping in scroll containers.
- **Theming:** sample palette from event logos so each PLE feels branded, not a generic dark dashboard.
- **Deploy packaging:** junction logos failed on Linux builders; real files + correct ignore patterns required.

4. Interview Prep — Elevator & Behavioral

60-second elevator pitch

“I built a full-stack analytics platform for WWE Premium Live Events from 2000–2027. It starts with a documented collection pipeline into a dimensional warehouse and analytics marts, exposes them through FastAPI, and ships a React + Plotly dashboard with event-logo theming. It’s deployed on Vercel with the UI and Python API in one project. The focus was trustworthy metrics, clear product UX, and production packaging — not just charts on a CSV.”

STAR stories to prepare

Q. Tell me about a data-quality decision you made.

Attendance nulls were being treated like zero and inflated or collapsed averages. I excluded non-positive attendance from average calculations and added a yearly average view so the metric matched user intent. Constraint: never invent attendance.

Q. Describe a production deploy issue.

Event logos were a Windows junction and later excluded by a broad `logos` vercelignore rule. Linux build couldn’t see them (404s in prod). Fixed by shipping real files under `frontend/public/logos` and scoping ignore to root-only `/logos`, then redeploying.

Q. How did you handle conflicting ‘truth’ between dataset and public records?

PLE-scoped marts can’t always reproduce all-time records (e.g. Cena’s 17). Timeline exposes curated landmark records with clear framing, while warehouse charts remain honest to PLE data — transparency over fake equality.

Behavioral one-liners

- Owned product + engineering: filters, themes, API contracts, deploy.
- Prefer constraints over cleverness: eligibility registry, no fabricated rows.
- Iterate with user-visible bugs (avatar gender labels, filter portal scroll, Crown Jewel titles).

5. Technical Q&A (Practice Answers)

System design & data

Q. Why marts instead of querying the warehouse from the UI?

Dashboards need stable, fast, denormalized slices. Marts keep the serverless API simple (read CSV/pandas), reduce join cost, and freeze metric definitions so the React app doesn't reimplement business logic.

Q. Why FastAPI?

Native async ASGI (Vercel-friendly), automatic OpenAPI, Pydantic validation, and low boilerplate for read-heavy analytics endpoints.

Q. How do you prevent weekly Raw results from polluting PLE analytics?

Eligibility config + series registry. Collectors and warehouse loads only include Premium Live Events and allowlisted specials — Raw/SmackDown/NXT TakeOver/house shows are excluded by rule.

Q. How does wrestler deduplication work at a high level?

Normalize display names, map aliases, and collapse duplicate identities when serving roster endpoints so the UI shows one career card instead of fragmented rows.

Q. What would you change at larger scale?

Move marts to a columnar store or Postgres, add caching (CDN/ETag) on API GETs, paginate heavy endpoints, split logo assets to object storage/CDN, and CI-build marts on a schedule instead of shipping CSVs in the function bundle.

Frontend & product

Q. Why Plotly over Chart.js / Recharts?

Rich statistical chart types, hover templates, and fast iteration for analytics-heavy pages without building a custom viz layer.

Q. Explain the event theme system.

Each event logo contributes a palette; CSS variables drive accents so WrestleMania vs Summerslam feel distinct while sharing one design system (broadcast cards, KPI strips, tables).

DevOps / Vercel

Q. How is the API packaged for Vercel?

`api/index.py` adds `src` to `sys.path` and exports the FastAPI `app`. Requirements stay lean in `api/requirements.txt`. Marts and runtime export CSVs are included via `includeFiles` because primary exports are gitignored.

Q. SPA + API on one domain — how?

Rewrites send `/api/*` to the Python function; other non-file routes fall through to `index.html` for client routing.

6. Interview Cheat Sheet

Topic	Soundbite
Problem	Trusted PLE analytics UI over messy multi-source wrestling history
Approach	Registry → clean → dimensional WH → marts → API → React
Stack	Python/FastAPI/Pandas + React/TS/Vite/Plotly + Vercel
Quality	No fabricated rows; null ≠ 0; PLE-scoped titles
UX wins	Logo themes, sticky filters, full deduped roster, Timeline landmarks
Prod	https://wwe-anaylsis.vercel.app
Code	https://github.com/alirazach476/wwe_anaylsis
Scale next	DB-backed marts, CDN assets, cache, CI refresh

Questions to ask the interviewer

- How does your team separate metric definitions from UI code (semantic layer / marts / metrics store)?
- What's your approach to historical data corrections without silent rewrites?
- For read-heavy dashboards, do you prefer BFF APIs, edge caches, or warehouse-direct tools?

WWE Anaylsis — Project Working · Tech Stack · Interview Prep
Generated for interview readiness · Live demo on Vercel